

07 — การคำนวณในทุกะดับ: step · episode · iteration · epoch · mini-batch — derive ด้วยมือ

ทุกคำ สัญลักษณ์ และเลขสมการ นิยามที่เดียวใน [00-glossary.md](#) — เลขสมการ (N) ในโน้ตนี้ คือเลขเดียวกับโน้ต 05 · ตัวเลขทุกตัวของตาราง 2×4 อยู่ในอภิธานศัพท์หัวข้อ 8 และคำนวณด้วย `scripts/toy_answer_key.py` (รับจากรากของ repo)

โน้ตนี้มีเป้าหมายเดียว: **ให้คุณอธิบายกระบวนการแทนทั้งหมดบนกระดาษได้ด้วยมือ โดยไม่ต้องเปิดโค้ด** — ทุกสูตรถูก derive จากสิ่งที่ง่ายกว่า และทุกตัวเลขคำนวณ ตามได้ด้วยเครื่องคิดเลข

วิธีทำ: ใช้ **ตัวอย่างขนาดจิ๋ว** — $2 \text{ environments} \times 4 \text{ steps} \times \text{action } 1 \text{ มิติ} = 8 \text{ transitions}$ — ที่เลือกจะเขียนทุกตัวเลขลงกระดาษ แล้วค่อยสเกลกลับไปขนาดจริง ($2048 \times 24 \times 8 \text{ มิติ}$) ด้วยตารางเทียบตอนท้าย ตัวเลขทุกตัวในโน้ตนี้ตรวจด้วย Python แล้ว และสูตรทุกสูตรตรงกับโค้ดใน `tron1-rl-isaacsim/rs1_rl/` แล้ว

โดอะแกรมประกอบ: [diagrams/tron1-five-units.html](#) — ตาราง 24×2048 ที่เป็น "ข้อมูลหนึ่ง iteration" และห้าหน่วยนับคือห้าวิธีมองมัน

0. ห้าหน่วยนับคือห้าวิธีเห็นตารางเดียวกัน

ก่อนจะคำนวณอะไร ต้องเห็นภาพนี้ให้ชัดก่อน — **ข้อมูลของหนึ่ง iteration คือตาราง สองมิติ** แถว = step (24) คอลัมน์ = environment (2048) แต่ละช่อง = หนึ่ง transition ($s, a, r, V, \log \pi, \text{done}$)

	env 0	env 1	env 2	...	env 2047	← 2048 คอลัมน์
step $t = 0$	▪	▪	▪	...	▪	24 แถว
step $t = 1$	▪	▪	▪	...	▪	
step $t = 2$	▪	▪	▪	...	▪	
⋮	⋮	⋮	⋮	⋮	⋮	
step $t = 23$	▪	▪	▪	...	▪	

= 49,152 ช่อง

▪ = 1 step	= 1 ช่อง
episode	= ช่วงต่อเนื่องในแนวตั้งของ 1 คอลัมน์ จนถึงช่องที่ <code>done=1</code> (อาจสั้นกว่า 24 หรือยาวข้ามหลายตาราง)
iteration	= ตารางทั้งใบ → GAE ครั้งเดียว → เรียนรู้ → ล้าง → ตารางใหม่
epoch	= การเดินผ่านทุกช่องในตารางครบ 1 รอบ
mini-batch	= ช่องที่สุ่มมา 1/4 ของตาราง (12,288 ช่อง) → 1 gradient step

หน่วย	คืออะไรในตาราง	คำนวณอะไร "ที่ระดับนี้"	ใน toy	ใน Tron1
step	1 ช่อง	forward pass: $\mu, a, \log \pi, V$ · env: r, done	8 ช่อง	49,152 ช่อง
episode	ช่วงแนวตั้งใน 1 คอลัมน์	ไม่มีสูตร — แค่ reset env และจดความยาว แต่ <code>done</code> ตัดสาย GAE	e1 มี 2 episode	~1000 steps ≈ 42 ตาราง
iteration	ตารางทั้งใบ	GAE + normalize ครั้งเดียว ทุกช่องพร้อมกัน แล้ว epoch × minibatch gradient steps แล้ว KL → lr	1 ตาราง 2×4	1 ตาราง 24×2048
epoch	เดินครบทุกช่อง	สับลำดับช่อง → หั่นเป็น minibatch	2 epoch × 2 mb	5 epoch × 4 mb
mini-batch	ช่องสุ่ม 1/M ของตาราง	$\rho, L^{CLIP}, L_V, H \rightarrow 1 \text{ gradient step}$	4 ช่อง	12,288 ช่อง

สองสิ่งที่คนมักเข้าใจผิด:

1. **episode ไม่ใช่หน่วยของอัลกอริทึม** — PPO ไม่เคยรอ episode จบ ไม่เคยคำนวณ อะไร "ต่อ episode" มันเป็นหน่วยของ *environment* (เมื่อไหร่ต้อง reset) และของ *การวัดผล* (Mean episode length) เท่านั้น สิ่งเดียวที่ episode มีผลต่อคณิตศาสตร์ คือ done ที่ตัดสาย bootstrap ใน GAE (§4.2)
2. **gradient step เกิดที่ระดับ mini-batch ไม่ใช่ iteration** — 1 iteration มี $5 \times 4 = 20$ gradient steps; GAE ต่างหากที่เกิดครั้งเดียวต่อ iteration

1. ตัวอย่างจิ้ง — ตั้งค่าทั้งหมดที่จะใช้

เพื่อให้คำนวณด้วยมือได้ ใช้ค่าคงที่กลม ๆ (ต่างจาก Tron1 จริง แต่สูตรเดียวกันเป๊ะ):

ค่าคงที่	toy	Tron1 จริง	ความหมาย	ค่า Tron1 มาจากบัสสัท
B (envs)	2	2048	จำนวนคอลัมน์	CLI <code>--num_envs 2048</code> (../README.md:265); default ใน <code>cfg</code> 4096 <code>cfg/SF/limx_base_env_cfg.py:484</code>
T (steps)	4	24	จำนวนแถว	<code>num_steps_per_env</code> <code>limx_rsl_rl_ppo_cfg.py:81</code>
action dim	1	8	มิติของ $\mathbf{a}$ — ใช้ 1 เพื่อให้ $\log \pi$ มีพจน์เดียว	<code>ActionsCfg</code> <code>cfg/SF/limx_base_env_cfg.py:116-121</code> → 8 ข้อต่อ <code>solefoot_cfg.py:41-57</code>
γ	0.9	0.99	discount	<code>gamma</code> <code>cfg:102</code>
λ	0.8	0.95	GAE	<code>lam</code> <code>cfg:103</code>
$\gamma\lambda$	0.72	0.9405	ตัวถ่วงน้ำหนัก GAE — คำนวณล่วงหน้าไว้	—
σ	0.5 (คงที่, $\ell = -0.6931$)	เริ่ม 1.0 ค่อย ๆ ลด	ความกว้างการสุ่ม	<code>logstd = zeros</code> <code>actor_critic.py:118</code>
ϵ (clip)	0.2	0.2	ขอบ clip	<code>clip_param</code> <code>cfg:96</code>
epochs $\times$ minibatches	2×2	5×4	gradient steps ต่อ iteration = 4 vs 20	<code>cfg:98-99</code>

ข้อมูลดิบในตาราง 2×4 — สิ่งที่ "network คาย" (μ, V) · "ลูกเต๋าของ policy" (ϵ — สุ่มใน `.sample()` ไม่ใช่โลกให้) · "โลกให้มา" (r , done) ระหว่างช่วงเก็บข้อมูล (สมมติขึ้นให้ครบทุกกรณี: e1 ล้มที่ t=2):

		t=0	t=1	t=2	t=3	V_{last}
env 0	μ (actor คาย)	0.20	0.30	0.10	0.40	
	ϵ (สุ่ม)	+0.40	-0.20	+0.80	0.00	
	r (reward)	1.0	0.5	1.0	0.5	
	V (critic คาย)	5.0	4.5	4.0	3.5	3.0
	done	0	0	0	0	
env 1	μ	-0.10	0.00	0.20	-0.30	
	ϵ	-0.60	+0.80	+0.20	+0.40	
	r	0.5	1.0	-2.0	0.0	
	V	3.0	2.0	1.0	4.0	3.5
	done	0	0	1	0	

env 1 ล้มหลัง action ที่ t=2 (reward -2, done=1) แล้ว reset — ช่วง t=3 ของ env 1 จึงเป็น **episode ใหม่** ($V = 4.0$ กระโดดขึ้นเพราะ ยืนตรงอีกครั้ง) นี่คือเคสที่ทำให้เห็นว่า done ทำอะไรกับ GAE · ส่วน $\epsilon = +0.80$ ที่ e0 t2 ตั้งใจให้ $\sigma\epsilon = +0.40$ เท่ากับของ e1 t1พอดี — สองช่วงนี้จึงเป็น **คู่แฝด** ที่ห่างจากจุดกลางเท่ากัน (§2.1) และ §5.3 ใช้คู่นี้ดู clip ครบทุกกรณี

2. ระดับ STEP — สิ่งที่ต้องคำนวณต่อหนึ่งช่วง

ทุกช่องทำสื่ออย่างนี้ **พร้อมกันทั้ง B คอลัมน์** (tensor operation ครั้งเดียว):

2.1 สุ่ม action — สมการ (16) ของไม้ต 05

$$a = \mu + \sigma \varepsilon$$

ช่อง	μ	$\sigma \varepsilon$	a
e0 t0	0.20	$0.5 \times (+0.40) = +0.20$	+0.40
e0 t1	0.30	$0.5 \times (-0.20) = -0.10$	+0.20
e0 t2	0.10	$0.5 \times (+0.80) = +0.40$	+0.50
e0 t3	0.40	$0.5 \times (0.00) = 0.00$	+0.40
e1 t0	-0.10	$0.5 \times (-0.60) = -0.30$	-0.40
e1 t1	0.00	$0.5 \times (+0.80) = +0.40$	+0.40
e1 t2	0.20	$0.5 \times (+0.20) = +0.10$	+0.30
e1 t3	-0.30	$0.5 \times (+0.40) = +0.20$	-0.10

2.2 $\log \pi_{old}(a | s)$ — derive จาก Gaussian PDF

เริ่มจากสมการ (13): $p(a) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(a-\mu)^2}{2\sigma^2}\right)$

ใส่ **log** ทั้งสองข้าง แล้วใช้กฎ $\log(xy) = \log x + \log y$ และ $\log e^u = u$:

ขั้น	ผล
$\log p = \log\left(\frac{1}{\sigma\sqrt{2\pi}}\right) + \log \exp(\dots)$	แยกผลคูณเป็นผลบวก
$= -\log \sigma - \log \sqrt{2\pi} - \frac{(a-\mu)^2}{2\sigma^2}$	$\log \frac{1}{x} = -\log x$ และ $\log \sqrt{2\pi} = \frac{1}{2} \log 2\pi$

$$\log \pi(a | s) = -\frac{1}{2} \ln 2\pi - \ln \sigma - \frac{(a-\mu)^2}{2\sigma^2} \quad (14)$$

สองพจน์แรกคงที่ทั้งตาราง (เพราะ σ ไม่ขึ้นกับ state): $-\frac{1}{2} \ln 2\pi = -0.9189$ และ $-\ln 0.5 = +0.6931$ รวม = **-0.2258** เหลือแค่พจน์สุดท้ายที่ต่างกันต่อช่อง:

ช่อง	$(a-\mu)$	$(a-\mu)^2/(2\sigma^2) = (a-\mu)^2/0.5$	$\log \pi_{old} = -0.2258 - \text{พจน์นี้}$
e0 t0	0.20	0.0800	-0.3058
e0 t1	-0.10	0.0200	-0.2458
e0 t2	0.40	0.3200	-0.5458
e0 t3	0.00	0.0000	-0.2258
e1 t0	-0.30	0.1800	-0.4058
e1 t1	0.40	0.3200	-0.5458
e1 t2	0.10	0.0200	-0.2458
e1 t3	0.20	0.0800	-0.3058

สังเกต: e0 t3 ที่ $\varepsilon = 0$ (สุ่มได้ตรง μ พอดี) มี **log π** สูงสุด = -0.2258 = ค่าที่ยอดระยะยัง a ห่างจาก μ ยิ่งติดลบมาก — ตรงกับตารางลดก่อน ในไม้ต 05 หัวข้อ 2.3 · คำนี้อีกถูกเก็บลง **buffer** เพื่อใช้เป็น "ค่าอ้างอิงจากอดีต" ตอนคำนวณ ρ ใน §5 — ในโค้ดคือ [ppo.py:147](#)

2.3 $V(s)$ และ r , done

V มาจาก critic forward pass (สมการ (10) ข้อ 4 ขั้น) — ในตัวอย่างนี้ให้มาแล้ว · r กับ done มาจาก `env.step()` หลังส่ง $a \times 0.25$ เข้า PD (ไม้ต 05 หัวข้อ 2.5)

สรุประดับ step: หนึ่งช่องเก็บ 6 ค่า ($s, a, r, V, \log \pi_{old}, \text{done}$) ทำ 4 ครั้ง ($t=0..3$) $\times$ 2 คอลัมน์ = 8 ช่องเต็ม → จบช่วง A

3. ระดับ EPISODE — ไม่มีสูตร แต่มีผลข้างเคียงหนึ่งอย่าง

env 1 มี 2 episode ในตารางนี้: $\{t_0, t_1, t_2\}$ (จบเพราะล้ม) และ $\{t_3, \dots\}$ (ต่อไปยังตารางถัดไป) · env 0 มี episode เดียวที่ยังไม่จบ (ต่อไปตารางหน้า)

สิ่งที่ episode "ทำ":

อะไร	ทำโดยใคร	สูตรใหม่
reset env กลับทำเริ่มต้น	environment	ไม่
จดความยาว → Mean episode length	logger	ไม่ (แค่นับ)
จดผลรวม reward → Mean reward	logger	ไม่ (แค่นับ)
ตั้ง done=1 ที่ช่องสุดท้าย	environment	มีผลต่อ GAE ใน §4.2

นั่นคือทั้งหมด — ไม่มีสมการไหนในไม้ต 05 ที่คำนวณ "ต่อ episode" สมการ (2) และ (3) พูดยังผลรวมถึง T ก็จริง แต่ในทางปฏิบัติถูกแทนด้วย bootstrap (§4.1) ทำให้ ไม่ต้องรอ episode จบเลย

Remark — done มีสองสาเหตุ และได้ปฏิบัติต่างกัน. ตารางนี้มีแต่กรณีล้ม (base_contact cfg/SF/limx_base_env_cfg.py:451-454) ซึ่งอนาคตถูกตัดจริง mask ใน §4.2 จึงตัดพจน์ bootstrap ทั้ง ถูกต้อง · แต่ หมดเวลา (time_out :450, 1000 step) อนาคตยังมี — ppo.py:163-168 จึงบวก $\gamma \cdot V(s_t)$ กลับเข้าไปใน r_t ก่อนเก็บลง buffer (ใช้ V ของช่องนั้นเองที่ critic เพิ่งคำนวณ แทน $V(s_{t+1})$ ที่ไม่ได้คำนวณ — การประมาณของ rsl_rl) · ถ้า e1 t2 เป็นหมดเวลา แทนล้ม ค่า r ในตารางจะกลายเป็น $-2.0 + 0.9 \times 1.0 = -1.1$ ก่อนที่ §4 จะเห็นมัน

4. ระดับ ITERATION (ส่วนที่ทำครั้งเดียว) — GAE ทั้งตาราง

ตอนนี้ตารางเต็มแล้ว ก่อนจะเรียนรู้ ต้องแปลง (r, V) ทุกช่องให้เป็น "เครดิต" A ทำครั้งเดียว ทุกคอลัมน์พร้อมกัน ไล่จากแถวล่างขึ้นบน

4.1 Bootstrap — คำที่ "ขอบตาราง"

แถวสุดท้าย (t=3) ไม่มีแถว t=4 ให้ V ถัดไป → ให้ critic ทำนาย $V(s_4)$ จาก state หลังสุด: $V_{last} = 3.0$ (e0), 3.5 (e1) — นี่คือการ bootstrapping ตามไม้ต 05 หัวข้อ 3.2: ใช้ค่าประมาณแทนการรอผลจริง

4.2 TD-error δ_t — derive จากสมการ Bellman

สมการ (4): $V(s_t) = \mathbb{E}[r_t + \gamma V(s_{t+1}) \mid s_t]$ บอกว่า ถ้า critic แม่น ข้างซ้ายเท่าข้างขวา · ส่วนต่างระหว่างสองข้างจึงวัดว่า critic เดาพลาดเท่าไร:

$$\delta_t = r_t + \underbrace{(1 - \text{done}_t)}_{\text{mask}} \gamma V(s_{t+1}) - V(s_t)$$

(19)

พจน์ $(1 - \text{done}_t)$ คือสิ่งที่โค้ดเรียก next_is_not_terminal — ถ้า episode จบ ที่ช่องนี้ ไม่มี s_{t+1} ที่มีความหมาย (env reset ไปแล้ว) จึงต้องตัดพจน์ bootstrap ทั้ง ไล่จาก t=3 ลงมา:

ช่อง	r_t	mask	$\gamma \cdot V_{t+1}$	V_t	δ_t
e0 t3	0.5	1	$0.9 \times 3.0 (=V_{last}) = 2.70$	3.5	$0.5 + 2.70 - 3.5 = -0.30$
e0 t2	1.0	1	$0.9 \times 3.5 = 3.15$	4.0	+0.15
e0 t1	0.5	1	$0.9 \times 4.0 = 3.60$	4.5	-0.40
e0 t0	1.0	1	$0.9 \times 4.5 = 4.05$	5.0	+0.05
e1 t3	0.0	1	$0.9 \times 3.5 (=V_{last}) = 3.15$	4.0	-0.85
e1 t2	-2.0	0	$0 \times 0.9 \times 4.0 = 0$	1.0	$-2.0 + 0 - 1.0 = -3.00$
e1 t1	1.0	1	$0.9 \times 1.0 = 0.90$	2.0	-0.10
e1 t0	0.5	1	$0.9 \times 2.0 = 1.80$	3.0	-0.70

กฎ e1 t2: ถ้าไม่มี mask จะได้ $\delta = -2 + 3.6 - 1 = +0.6$ — บอกว่า "ลื้ม แล้วดีกว่าที่คาด" ซึ่งผิดสิ้นเชิง เพราะ 3.6 นั้นคือ V ของ *ท้ายใหม่หลัง reset* ไม่ใช่อนาคตของค่าที่กำลังลื้ม mask ทำให้ได้ -3.0 ที่ถูกต้อง: ลื้มคือแย่มาก

4.3 Advantage A_t — derive ว่า recursion กับ closed-form คือสิ่งเดียวกัน

ใบ้ 05 สมการ (25) เขียน GAE เป็นผลรวม: $A_t = \sum_{k \geq 0} (\gamma \lambda)^k \delta_{t+k}$ แต่ได้เขียนเป็น **recursion**:

$$A_t = \delta_t + (1 - \text{done}_t) \gamma \lambda A_{t+1}, \quad A_T = 0 \quad (26)$$

พิสูจน์ว่าเท่ากัน (ไม่มี done): แทน A_{t+1} ด้วยนิยามของมันเองซ้ำ ๆ

ขั้น	ผล
$A_t = \delta_t + \gamma \lambda A_{t+1}$	นิยาม
$= \delta_t + \gamma \lambda (\delta_{t+1} + \gamma \lambda A_{t+2})$	แทน A_{t+1}
$= \delta_t + \gamma \lambda \delta_{t+1} + (\gamma \lambda)^2 A_{t+2}$	กระจาย
$= \delta_t + \gamma \lambda \delta_{t+1} + (\gamma \lambda)^2 \delta_{t+2} + (\gamma \lambda)^3 A_{t+3}$	แทนอีกครั้ง
$= \sum_{k=0}^{T-1-t} (\gamma \lambda)^k \delta_{t+k}$	ทำจนถึง $A_T = 0$ ■

recursion จึงเป็นแค่วิธีคำนวณผลรวมนั้นให้เร็ว (คูณบวกทีละก้าวแทนที่จะยกกำลังใหม่ ทุกครั้ง) และ mask $(1 - \text{done})$ ทำให้ผลรวมหยุดที่ขอบ episode

คำนวณด้วย recursion ($\gamma \lambda = 0.72$, เริ่ม $A_4 = 0$):

ช่วง	δ_t	mask $\times 0.72 \times A_{t+1}$	A_t
e0 t3	-0.30	$1 \times 0.72 \times 0 = 0$	-0.3000
e0 t2	+0.15	$1 \times 0.72 \times (-0.30) = -0.2160$	-0.0660
e0 t1	-0.40	$1 \times 0.72 \times (-0.066) = -0.0475$	-0.4475
e0 t0	+0.05	$1 \times 0.72 \times (-0.4475) = -0.3222$	-0.2722
e1 t3	-0.85	$1 \times 0.72 \times 0 = 0$	-0.8500
e1 t2	-3.00	$0 \times 0.72 \times (-0.85) = 0$	-3.0000
e1 t1	-0.10	$1 \times 0.72 \times (-3.0) = -2.1600$	-2.2600
e1 t0	-0.70	$1 \times 0.72 \times (-2.26) = -1.6272$	-2.3272

ตรวจด้วย closed-form (ทำเองบนกระดาษได้):

$$A_0^{(e0)} = \delta_0 + 0.72 \delta_1 + 0.72^2 \delta_2 + 0.72^3 \delta_3 = 0.05 - 0.288 + 0.0778 - 0.1120 = -0.2722 \checkmark$$

$$A_0^{(e1)} = \delta_0 + 0.72 \delta_1 + 0.72^2 \delta_2 + \underbrace{0 \cdot \delta_3}_{\text{mask ตัด}} = -0.70 - 0.072 - 1.5552 = -2.3272 \checkmark$$

สังเกต e1: $\delta_3 = -0.85$ ไม่ถูกนับใน A_0 เพราะเป็นคนละ episode — ถ้าไม่มี mask จะเอาความลื้มเหลวของ episode หน้าไปโทษ action ใน episode ก่อน

4.4 Return R_t — เป้าที่ critic ต้องเรียนรู้

$$R_t = A_t + V_t \quad (27)$$

ทำไม: จากนิยาม $A = (\text{ผลจริง} - (\text{ที่ ' critic เดา})) \rightarrow \text{ผลจริง} = A + V$ — นี่คือ "คำตอบเฉลย" ที่จะเอาไปสอน critic ใน §5.4

ช่วง	A_t	V_t	R_t
e0 t0	-0.2722	5.0	4.7278

ช่อง			
e0 t1	-0.4475	4.5	4.0525
e0 t2	-0.0660	4.0	3.9340
e0 t3	-0.3000	3.5	3.2000
e1 t0	-2.3272	3.0	0.6728
e1 t1	-2.2600	2.0	-0.2600
e1 t2	-3.0000	1.0	-2.0000
e1 t3	-0.8500	4.0	3.1500

4.5 Normalize advantage — ทำครั้งเดียวทั้งตาราง

$$\hat{A} = \frac{A - \text{mean}(A)}{\text{std}(A) + 10^{-8}} \quad (28)$$

ทำไม: ทำให้ขนาดของ A อยู่ราว ๆ ± 1 เสมอไม่ว่า reward จะใหญ่หรือเล็ก → learning rate ตัวเดียวใช้ได้ทุกงาน · จำนวนจากทั้ง 8 ค่าพร้อมกัน (ทั้ง 49,152 ใน Tron1):

$$\text{mean} = \frac{-0.2722 - 0.4475 - 0.0660 - 0.3000 - 2.3272 - 2.2600 - 3.0000 - 0.8500}{8} = -1.1904$$

$$\text{std} = \sqrt{\frac{\sum (A_i - \text{mean})^2}{8 - 1}} = 1.1513$$

กับดัก: `torch.std()` หารด้วย $n - 1$ (unbiased) ไม่ใช่ n — ถ้าคุณคำนวณด้วยมือ แล้วได้ 1.077 แปลว่าหารด้วย 8 ต้องหารด้วย 7 ถึงจะตรงกับโค้ด

ช่อง	A	$\hat{A} = (A + 1.1904)/1.1513$
e0 t0	-0.2722	+0.7975
e0 t1	-0.4475	+0.6452
e0 t2	-0.0660	+0.9766
e0 t3	-0.3000	+0.7733
e1 t0	-2.3272	-0.9874
e1 t1	-2.2600	-0.9290
e1 t2	-3.0000	-1.5718
e1 t3	-0.8500	+0.2956

อ่านผล: ทุก action ของ e0 กลายเป็น "ดีกว่าค่าเฉลี่ย" (บวก) และ e1 t0-t2 เป็น "แยกว่า" (ลบ) — แม้ A ดิบของ e0 จะติดลบทั้งหมด! เพราะ normalize วัตถุประสงค์ **เพื่อนในตารางเดียวกัน** ไม่ใช่เทียบกับศูนย์ นี่คือเหตุผลที่ไม้ 05 หัวข้อ 4.5 เตือนว่า PPO ไม่ได้ดู reward ดิบ

สรุประดับ iteration (ครั้งเดียว): ตารางตอนนี้มี $\hat{A}$ และ R ครบทุกช่อง พร้อม สำหรับการเรียนรู้ — ในโค้ดคือ [rollout_storage.py:187-206](#)

5. ระดับ EPOCH และ MINI-BATCH — ที่ที่ weight เปลี่ยน

5.1 หั่นตาราง

8 ช่อง → สับลำดับ**ครั้งเดียว** (`torch.randperm rollout_storage.py:230` อยู่นอก loop epoch :251) → หั่นเป็น 2 minibatch ละ 4 ช่อง · เดินครบทั้ง 2 = 1 epoch · ทำ 2 epoch ด้วยการหั่นชุดเดิม = **4 gradient steps ต่อ iteration**

สับครึ่งเดียว:	$mb_0 = \{e0t0, e0t2, e1t1, e1t3\}$	$mb_1 = \{e0t1, e0t3, e1t0, e1t2\}$
epoch 1:	$mb_0 \rightarrow \text{gradient step \#1}$	$mb_1 \rightarrow \text{gradient step \#2}$
epoch 2:	$mb_0 \rightarrow \text{gradient step \#3}$	$mb_1 \rightarrow \text{gradient step \#4}$ (ชุดเดิม ไม่สับใหม่)
		(Tron1: $5 \times 4 = 20$ steps)

จุดสำคัญ: ข้อมูล $(a, \log \pi_{old}, \hat{A}, R)$ ในตารางไม่เปลี่ยนตลอด 4 steps สิ่งที่เปลี่ยนคือ $\theta \rightarrow \mu_{new} \rightarrow \log \pi_{new} \rightarrow \rho$ ที่คำนวณใหม่ทุก step

5.2 หนึ่ง gradient step บน mb₀ — ไล่ทุกสูตร

สมมติหลัง gradient step ก่อนหน้า actor คาย μ ที่ต่างจากตอนเก็บข้อมูล เล็กน้อย: $\mu_{new} = \mu_{old} + 0.05$ ทุกช่อง (ในความจริง network คำนวณค่านี้ใหม่ จาก s เดิม แต่ด้วย weight ใหม่)

(ก) ρ — derive จากนิยาม

$$\rho = \frac{\pi_{new}(a \mid s)}{\pi_{old}(a \mid s)} = \frac{e^{\log \pi_{new}}}{e^{\log \pi_{old}}} = e^{\log \pi_{new} - \log \pi_{old}}$$

จึงไม่ต้องคำนวณ π ตรง ๆ แค่ **au log สองตัวแล้ว exp** — เสถียรกว่ามาก · $\log \pi_{new}$ ใช้สมการ (14) เดิมแต่แทน μ_{new} และ a ตัวเดิมจาก buffer:

ช่อง	a	μ_{new}	$(a - \mu_{new})^2 / 0.5$	$\log \pi_{new}$	$\log \pi_{old}$	Δ	$\rho = e^\Delta$
e0 t0	0.40	0.25	0.0450	-0.2708	-0.3058	+0.0350	1.0356
e0 t2	0.50	0.15	0.2450	-0.4708	-0.5458	+0.0750	1.0779
e1 t1	0.40	0.05	0.2450	-0.4708	-0.5458	+0.0750	1.0779
e1 t3	-0.10	-0.25	0.0450	-0.2708	-0.3058	+0.0350	1.0356

ทุก $\rho > 1$ เพราะ μ ขยับเข้าหา a ในทุกช่อง (บังเอิญจากที่เลือก +0.05) → policy ใหม่ "ชอบ" action เก่าเหล่านี้มากขึ้น 3-8%

(ข) Clipped surrogate — สมการ (29)

$$L_i = \max \left(-\rho_i \hat{A}_i, -\text{clip}(\rho_i, 0.8, 1.2) \hat{A}_i \right)$$

(เขียนเป็น loss ที่ต้องลด จึงติดลบจากรูปของเปเปอร์ และ **min** กลายเป็น **max** — ตรงกับ [ppo.py:256-260](#))

ช่อง	ρ	$\text{clip}(\rho)$	$\hat{A}$	$-\rho \hat{A}$	$-\text{clip}(\rho) \hat{A}$	$L_i = \max$	clip ทำงาน?
e0 t0	1.0356	1.0356	+0.7975	-0.8259	-0.8259	-0.8259	ไม่ (ρ ในช่วง)
e0 t2	1.0779	1.0779	+0.9766	-1.0526	-1.0526	-1.0526	ไม่
e1 t1	1.0779	1.0779	-0.9290	+1.0014	+1.0014	+1.0014	ไม่
e1 t3	1.0356	1.0356	+0.2956	-0.3062	-0.3062	-0.3062	ไม่

$$L^{CLIP} = \text{mean}(L_i) = \frac{-0.8259 - 1.0526 + 1.0014 - 0.3062}{4} = \textbf{-0.2958}$$

อ่าน: e1 t1 ให้ loss บวก (แย่) เพราะ $\hat{A} < 0$ (action นั้นแย่) แต่ $\rho > 1$ (policy ใหม่ดันชอบมันมากขึ้น) — gradient จะแก้ตรงนี้ · ยังไม่มี clip เพราะ ρ ทุกตัวอยู่ใน **[0.8, 1.2]** — ดู §5.3 ว่าเมื่อไหร่มันจะทำงาน

mb₁ — อีกสี่ช่อง ด้วยกฎ +0.05 เดียวกัน (มี $\rho < 1$ ให้ดู)

ช่อง	a	μ_{new}	$(a - \mu_{new})^2 / 0.5$	$\log \pi_{new}$	$\log \pi_{old}$	Δ	ρ	$\hat{A}$	L_i
e0 t1	0.20	0.35	0.0450	-0.2708	-0.2458	-0.0250	0.9753	+0.6452	-0.6293
e0 t3	0.40	0.45	0.0050	-0.2308	-0.2258	-0.0050	0.9950	+0.7733	-0.7695
e1 t0	-0.40	-0.05	0.2450	-0.4708	-0.4058	-0.0650	0.9371	-0.9874	+0.9253
e1 t2	0.30	0.25	0.0050	-0.2308	-0.2458	+0.0150	1.0151	-1.5718	+1.5955

$$L_{mb_1}^{CLIP} = \frac{-0.6293 - 0.7695 + 0.9253 + 1.5955}{4} = +0.2805$$

อ่าน: สามช่องแรกได้ $\rho < 1$ เพราะ a ไม่สูงกว่า μ_{old} (e0 t1: $0.20 < 0.30$ · e0 t3: $0.40 = 0.40$ พอดี) การขยับ $+0.05$ แบบเหมารวมจึงเป็นการ **หมี a** — policy ใหม่ชอบ action เก่าน้อยลง · นี่คือข้อจำกัดของ "กฎ $+0.05$ " ที่ใช้เพื่อให้คำนวณตามได้: gradient จริง (ค) จะขยับแต่ละช่องคนละทิศ · และ mb_1 ให้ L^{CLIP} เป็น**บวก** เพราะสองช่องของ env 1 ที่ $\hat{A} < 0$ มี $|\hat{A}|$ ใหญ่ — loss บวกแปลว่า "ตอนนี้ policy ยังชอบ action แย่พวกนั้นอยู่" gradient จะดันออก

(ค) Gradient $\partial L / \partial \mu$ — derive ด้วยมือ: หัวใจของ "ทำใน policy ถึงเรียนรู้"

เมื่อ ρ อยู่ในช่วง clip พจน์ที่ใช้คือ $-\rho \hat{A}$ · หาอนุพันธ์เทียบ μ_{new} (ค่าที่ actor คาย) ผ่าน chain rule:

ขั้น	เหตุผล
$\frac{\partial \log \pi}{\partial \mu} = \frac{\partial}{\partial \mu} \left[-\frac{(a - \mu)^2}{2\sigma^2} \right] = \frac{a - \mu}{\sigma^2}$	จาก (14) พจน์เดียวที่มี μ — คือสมการ (24) ของไม้ต 05 · อนุพันธ์ของ $-(a - \mu)^2$ คือ $+2(a - \mu)$
$\frac{\partial \rho}{\partial \mu} = \rho \cdot \frac{\partial \log \pi}{\partial \mu} = \rho \frac{a - \mu}{\sigma^2}$	เพราะ $\rho = e^{\log \pi_{new} - c}$ และ $\frac{d}{dx} e^f = e^f f'$
$\frac{\partial L}{\partial \mu} = -\hat{A} \frac{\partial \rho}{\partial \mu}$	$\hat{A}$ เป็นค่าคงที่ (มาจาก buffer)

$$\frac{\partial L}{\partial \mu} = -\hat{A} \rho \frac{a - \mu_{new}}{\sigma^2} \quad (G)$$

อ่านสมการ (G) ให้ได้ความหมาย — gradient descent ทำ $\mu \leftarrow \mu - \alpha \partial L / \partial \mu$:

กรณี	$\hat{A}$	$a - \mu$	$\partial L / \partial \mu$	μ เคลื่อนไป	แปลว่า
action ดี, a อยู่ขวาของ μ	+	+	−	ขวา (เข้าหา a)	"ทำแบบนี้ไปอีก"
action ดี, a อยู่ซ้ายของ μ	+	−	+	ซ้าย (เข้าหา a)	"ทำแบบนี้ไปอีก"
action แย่, a อยู่ขวาของ μ	−	+	+	ซ้าย (หมี a)	"อย่าทำแบบนี้"
action แย่, a อยู่ซ้ายของ μ	−	−	−	ขวา (หมี a)	"อย่าทำแบบนี้"

นี่คือ policy gradient ทั้งหมดในหนึ่งบัสครั้ง: ขยับ μ เข้าหา action ที่ได้ advantage บวก หักจาก action ที่ได้ advantage ลบ โดยแรงเป็นสัดส่วนกับ $|\hat{A}|$

ตัวเลขจริง ($\sigma^2 = 0.25$):

ช่อง	$\hat{A}$	ρ	$a - \mu_{new}$	$\partial L / \partial \mu = -\hat{A} \rho (a - \mu) / 0.25$	SGD $\alpha = 0.1$: $\mu \leftarrow \mu - 0.1 \partial L / \partial \mu$
e0 t0	+0.7975	1.0356	+0.15	−0.4955	0.25 → 0.2995 (เข้าหา $a=0.40$ ✓)
e0 t2	+0.9766	1.0779	+0.35	−1.4737	0.15 → 0.2974 (เข้าหา $a=0.50$ ✓)
e1 t1	−0.9290	1.0779	+0.35	+1.4020	0.05 → −0.0902 (หนีจาก $a=0.40$ ✓)
e1 t3	+0.2956	1.0356	+0.15	−0.1837	−0.25 → −0.2316 (เข้าหา $a=−0.10$ ✓)

จาก μ ไปถึง θ : ในเครือข่ายจริง μ ไม่ใช่ตัวแปรอิสระ — มันคือ output ของ MLP ดังนั้น $\partial L / \partial \theta = \partial L / \partial \mu \cdot \partial \mu / \partial \theta$ พจน์หลังคือ backpropagation ผ่าน 4 ชั้น (ไม้ต 05 หัวข้อ 2.2) ที่ PyTorch ทำให้ใน `loss.backward()` — สิ่งที่ได้ด้วยมือได้และ**ควร**เข้าใจคือพจน์แรก (G) เพราะมันคือ ส่วนที่บอกกว่า "เรียนรู้ไปทางไหน" · Adam ต่างจาก SGD ตรงที่ปรับขนาดก้าวต่อ weight อัตโนมัติ แต่**ทิศทาง**เดียวกัน

5.3 เมื่อไหร่ clip จะทำงาน — คำนวณจากคู่แฝดในตาราง

ในตัวอย่างข้างบน clip ไม่ทำงานเลยเพราะ μ ขยับแค่ 0.05 · ตารางนี้จึงมี **คู่แฝด** ไว้ให้ทดสอบ: **e0 t2** กับ **e1 t1** มี $|a - \mu| = 0.40$ เท่ากัน จึงมี $\log \pi_{old} = -0.5458$ เท่ากัน และเมื่อ μ ขยับเท่ากันก็ได้อ ρ เท่ากันเป๊ะ — ต่างกันแค่เครื่องหมายของ $\hat{A}$ (+0.9766 กับ −0.9290) · เท่านั้นที่เห็น clip ครบทั้งสี่กรณีจากตัวเลขในตารางเดียวกัน

กฎการขยับ: สมมติว่าผ่านไปแล้ว μ gradient step แล้ว μ เดินตามทิศที่สมการ (G) สั่ง จนสะสมได้

$\mu_{new} = \mu_{old} + \text{sign}(\hat{A}) \cdot \text{sign}(a - \mu_{old}) \cdot 0.15$ — เข้าหา a ถ้า action ดี หนีจาก a ถ้า action แย่ (ตารางสี่กรณีได้สมการ (G)) ·
ทั้ง e0 t2 และ e1 t1 มี $a > \mu_{old}$ ตัวคูณ $\text{sign}(a - \mu_{old})$ จึงเป็น $+1$ ทั้งคู่ และ "ถูกทาง" ของ e0 t2 คือ $+0.15$ ส่วนของ e1 t1 คือ -0.15 · แล้วลองกลับทิศดูด้วยว่าถ้า "เดินผิดทาง" โกลเท่ากัน clip จะทำอะไร

ช่อง	$\hat{A}$	ขยับ	μ_{new}	ρ	ในช่วง?	$-\rho\hat{A}$	$-\text{clip}(\rho)\hat{A}$	$L = \max$	$\partial L/\partial \mu$
e0 t2	+0.9766	ถูกทาง +0.15	0.25	1.2153	$X > 1.2$	-1.1869	-1.1719	-1.1719 (คงที่)	0
e1 t1	-0.9290	ถูกทาง -0.15	-0.15	0.7520	$X < 0.8$	+0.6986	+0.7432	+0.7432 (คงที่)	0
e0 t2	+0.9766	ผิดทาง -0.15	-0.05	0.7520	$X < 0.8$	-0.7344	-0.7813	-0.7344	-1.6157 ไหล
e1 t1	-0.9290	ผิดทาง +0.15	0.15	1.2153	$X > 1.2$	+1.1291	+1.1148	+1.1291	+1.1291 ไหล

อ่านสองแถวบน: พอ ρ หลุดกรอบ $[0.8, 1.2]$ ใน **ทิศที่ถูก max** เลือกพจน์ที่ถูก clip ซึ่งเป็นค่าคงที่ →อนุพันธ์ = 0 → **หยุดผลัก** · ระบบพูดว่า "เข้าหา/หนี action นั้นมา 20% แล้ว พอ รอข้อมูลใหม่"

อ่านสองแถวล่าง: ρ หลุดกรอบเท่ากันเป๊ะ แต่ใน **ทิศที่ผิด — max** กลับเลือกพจน์ดิบ gradient ไหลเต็ม และเครื่องหมายของมันดึง μ กลับ · clip ไม่เคยขวางการแก้ผิด

ปลายการเรสู σ เล็ก clip จะทำงานถี่ขึ้นมาก — จาก (14) สองครั้งได้ $\rho = \exp\left(\frac{(a-\mu_{old})^2 - (a-\mu_{new})^2}{2\sigma^2}\right)$ · ถ้า $\sigma = 0.25$ การขยับ +0.15 เดียวกันบน e0 t2 ($a - \mu_{old} = 0.40, a - \mu_{new} = 0.25$) ให้ $\rho = \exp\left(\frac{0.40^2 - 0.25^2}{2 \cdot 0.25^2}\right) = \exp(0.78) = 2.18$ เกะๆ กรอบไปไกล · นี่คือการหมายเชิงปฏิบัติของ trade-off ในไม้ต 05 หัวข้อ 4.7 เรื่องจำนวน epoch

ตารางสรุปว่า clip ทำงานเมื่อไหร่ (ตรงกับตารางเดียวกันในไม้ต 05 หัวข้อ 4.6 (ก)):

$\hat{A}$	$\rho > 1 + \epsilon$ (ชอบมากขึ้น)	$\rho < 1 - \epsilon$ (ชอบน้อยลง)
+ (action ดี)	clip · gradient 0 — "ดีพอแล้ว หยุด"	ไม่ clip · gradient ดึงกลับ — "ทำไหมหนี action ดี"
- (action แย่)	ไม่ clip · gradient ผลักออก — "ทำไหมชอบ action แย่"	clip · gradient 0 — "หนีพอแล้ว หยุด"

clip ปิด gradient เฉพาะ**ทิศที่เคลื่อนไปถูกทางแล้วและไหลพอ** — ไม่เคยขวางการแก้ผิด

5.4 Value loss — สอน critic

เป้าคือ R จาก §4.4 · critic ค่ายค่าใหม่ V_{new} (weight ϕ เปลี่ยนไปแล้ว):

$$L_V = \text{mean}\left[\max\left((V_{new} - R)^2, (V_{clip} - R)^2\right)\right], \quad V_{clip} = V_{old} + \text{clip}(V_{new} - V_{old}, -0.2, 0.2) \quad (32)$$

ตัวอย่างช่อง e0 t0 ($V_{old} = 5.0, R = 4.7278$):

V_{new}	$(V_{new} - R)^2$	V_{clip}	$(V_{clip} - R)^2$	L_V ช่องนี้ = max
4.9	0.0297	$5.0 + \text{clip}(-0.1) = 4.9$	0.0297	0.0297
4.5	0.0519	$5.0 + \text{clip}(-0.5) = 4.8$	0.0052	0.0519 (เลือกตัวใหญ่)

max ทำให้ critic ถูก "ลงโทษแบบระวัง": ถ้ากระโดดไกล (-0.5) ก็ยังโดนคิดตามที่กระโดด จริง ไม่ได้ผ่อนลง · gradient $\partial L_V/\partial V = 2(V - R) \rightarrow$ ดัน V เข้าหา R

5.5 Entropy — derive สำหรับ Gaussian

นิยาม $H = -\mathbb{E}[\log p(a)]$ · แทน (14) แล้วใช้ $\mathbb{E}[(a - \mu)^2] = \sigma^2$:

$$H = -\mathbb{E}\left[-\frac{1}{2}\ln 2\pi - \ln \sigma - \frac{(a-\mu)^2}{2\sigma^2}\right] = \frac{1}{2}\ln 2\pi + \ln \sigma + \frac{\sigma^2}{2\sigma^2} = \boxed{\frac{1}{2} + \frac{1}{2}\ln 2\pi + \ln \sigma} \quad (31)$$

ที่ $\sigma = 0.5$: $H = 0.5 + 0.9189 - 0.6931 = 0.7258$ · ขึ้นกับ σ อย่างเดียว — ยิ่ง σ เล็ก H ยิ่งต่ำ (ติดลบได้) · พจน์ $-0.01H$ ใน loss (= บวกโบนัสนเข้า objective) จึงต้าน การหด σ (ไม้ต 05 หัวข้อ 4.6 ก)

5.6 รวม loss แล้วก้าว

$$L = L^{CLIP} + 1.0 L_V - 0.01 H \tag{33}$$

`loss.backward()` → gradient ทุก weight (θ ของ actor, ϕ ของ critic, logstd) · `clip_grad_norm_(1.0)` · `optimizer.step()` → **จบ 1 gradient step** · ทำอีก 3 ครั้ง (mb₁, แล้ว epoch 2) = จบช่วง C

5.7 KL → ปรับ lr — derive สำหรับ Gaussian

KL ระหว่าง $\mathcal{N}(\mu_o, \sigma_o)$ กับ $\mathcal{N}(\mu_n, \sigma_n)$ (สูตรมาตรฐาน, ตรงกับ [ppo.py:223-232](#) ทุกพจน์ — สมการ (30) ของโน้ต 05):

$$KL = \ln \frac{\sigma_n}{\sigma_o} + \frac{\sigma_o^2 + (\mu_o - \mu_n)^2}{2\sigma_n^2} - \frac{1}{2} \tag{30}$$

กรณี σ เท่ากัน สองพจน์ตัดกันเหลือ $\frac{(\Delta\mu)^2}{2\sigma^2}$ · ที่ $\Delta\mu = 0.05$: $KL = 0.0025/0.5 = \mathbf{0.0050}$

เทียบ `desired_kl = 0.01`: $0.005 < 0.02$ (ไม่ต้องลด lr) และ $0.005 = 0.005$ (พอดีขอบล่าง ไม่เพิ่ม) → lr คงเดิม · แต่ถ้าใช้กฎ ± 0.15 ของ §5.3: $KL = 0.0225/0.5 = \mathbf{0.045} > 0.02$ → lr ÷ 1.5 — ในชื่อนี้ ถ้าทำให้ใหญ่พอให้ clip ทำงาน ก็ใหญ่พอให้เบรกอัตโนมัติทำงานด้วย · แต่**ไม่เสมอไป**: clip ดูรายช่องและขึ้นกับ $|a - \mu|$ ส่วน KL ดูค่าเฉลี่ยทั้ง batch และขึ้นกับ $\Delta\mu/\sigma$ อย่างเดียว — action ที่สุ่มได้ไกลกว่า $\approx 1\sigma$ จะโดน clip ก่อนที่ KL จะแตะ 0.02

6. กลับสู่ขนาดจริง — ตารางเทียบ toy → Tron1

ทุกสูตรเหมือนเดิม เปลี่ยนแค่ขนาด:

ปริมาณ	toy	Tron1	ที่มา
ช่องในตาราง	$2 \times 4 = 8$	$2048 \times 24 = \mathbf{49,152}$	<code>num_envs × num_steps_per_env</code>
$\log \pi$ ต่อช่อง	1 พจน์	ผลรวม 8 พจน์ (8 ข้อต่อ อีซส)	<code>.sum(dim=-1)</code> — โน้ต 05 หัวข้อ 2.3
$\gamma\lambda$	0.72	0.9405	horizon 3.6 vs ≈ 17 step — สมการ (34) โน้ต 05 หัวข้อ 4.7
GAE	4 ก้าวย้อน × 2 คอลัมน์	24 ก้าวย้อน × 2048 คอลัมน์ พร้อมกัน	vectorized uu GPU
normalize	จาก 8 ค่า	จาก 49,152 ค่า	
minibatch	4 ช่อง	12,288 ช่อง	<code>49152 / num_mini_batches 4</code>
gradient steps / iteration	$2 \times 2 = 4$	$5 \times 4 = \mathbf{20}$	<code>num_learning_epochs × num_mini_batches</code>
$\partial L / \partial \mu$	1 ตัวเลขต่อช่อง	8 ตัวเลขต่อช่อง	ต่อข้อต่อ
$\partial L / \partial \theta$	—	187,280 ตัวเลข (actor 187,272 + ℓ_8) · ทั้ง optimizer หลัก 469,905	backprop ผ่าน MLP
ขนาดของ V, δ, L_V ต่อช่อง	$3 \cdot 5 \cdot \pm 3 \cdot \sim 0.05$	$\sim 650 \cdot \sim 28 \cdot \sim 781$	ตัวเลขประกอบจากเอกสารต้นทาง (<code>sources/</code>) ไม่ใช่ค่าจริง — reward ของ Tron1 รวม 20 พจน์ต่อ step และ episode ยาว 1000 step ค่า V จึงใหญ่กว่า toy สองร้อยเท่า
iterations	1	15,000	ตารางใหม่ 15,000 ใบ
เวลาจริงต่อ iteration	—	~ 0.6 s (collection 0.42 + learning 0.18)	ตัวอย่างรูปแบบลือ ของ <code>train.py</code> — ยังไม่ได้วัดจริง (exp_001 ยังไม่รัน)

7. ทุกสูตรในหน้าเดียว — สำหรับเขียนบนกระดาน

เรียงตามลำดับที่เกิดจริง พร้อมระดับที่มันอยู่:

เลขในคอลัมน์ "สมการ" คือเลขเดียวกับโน้ต 05 และอภิธานศัพท์หัวข้อ 9 — ตารางนี้คือดัชนีนั้น มองตามระดับ ไม่ใช่สำเนาชุดที่สี่:

#	ระดับ	สูตร	สมการ	อ่านว่า
1	step	$a = \mu_\theta(s) + \sigma \varepsilon, \varepsilon \sim \mathcal{N}(0, 1)$	(9) (16)	สุ่มรอบค่าที่ actor คิดว่าดี
2	step	$\log \pi = -\frac{1}{2} \ln 2\pi - \ln \sigma - \frac{(a-\mu)^2}{2\sigma^2}$	(14) (15)	เก็บไว้เป็นค่าอ้างอิง
3	step	$V = V_\phi(s) \cdot r, \text{done} = \text{env}(a)$	(3) (10) · (17) (18)	critic เอา · โลกตอบ
4	iteration	$V_{\text{last}} = V_\phi(s_T)$	— (bootstrap)	bootstrap ที่จบตาราง
5	iteration	$\delta_t = r_t + (1 - d_t)\gamma V_{t+1} - V_t$	(19)	critic พลาดเท่าไร
6	iteration	$A_t = \delta_t + (1 - d_t)\gamma \lambda A_{t+1}$	(26) $\equiv$ (25)	เครดิตสะสม ย้อนหลัง
7	iteration	$R_t = A_t + V_t$	(27)	เป้าของ critic
8	iteration	$\hat{A} = (A - \bar{A}) / \text{std}_{n-1}(A)$	(28)	เทียบกับเพื่อนในตาราง
9	minibatch	$\rho = \exp(\log \pi_{\text{new}} - \log \pi_{\text{old}})$	(21)	ชอบมากขึ้นกี่เท่า
10	minibatch	$L^{\text{CLIP}} = \text{mean max}(-\rho \hat{A}, -\text{clip}(\rho) \hat{A})$	(29)	ให้รางวัลแต่ไม่เกิน 20%
11	minibatch	$L_V = \text{mean max}((V - R)^2, (V_{\text{clip}} - R)^2)$	(32)	สอน critic
12	minibatch	$H = \frac{1}{2} + \frac{1}{2} \ln 2\pi + \ln \sigma$	(31)	ยังสำรวจอยู่ไหม
13	minibatch	$L = L^{\text{CLIP}} + L_V - 0.01 H \cdot \theta \leftarrow \theta - \alpha \nabla_\theta L$	(33) (12) · (36) Adam	ก้าวหนึ่งก้าว
14	minibatch	$\partial L / \partial \mu = -\hat{A} \rho (a - \mu) / \sigma^2$	(G) จาก (24) · (35) สำหรับ ℓ	ทิศทางการเรียนรู้
15	minibatch	$KL = \ln \frac{\sigma_n}{\sigma_o} + \frac{\sigma_o^2 + (\Delta\mu)^2}{2\sigma_n^2} - \frac{1}{2} \rightarrow \text{lr} \times / \div 1.5$	(30) (37)	เบรคอัตโนมัติ
—	episode	(ไม่มี) — แต่ d_t ในสูตร 5-6	—	reset + วัตถุประสงค์

ถ้าต้องอธิบายให้คนอื่นใน 3 ประโยค: (1) ทุก step สุ่ม action รอบ μ แล้วจดว่า "ตอนนั้นชอบมันแค่ไหน" ($\log \pi_{\text{old}}$) (2) ครบตารางแล้ว ให้เครดิตแต่ละ action ย้อนหลังด้วย GAE เทียบกับที่ critic คาด (3) แล้วยับ μ เข้าหา action ที่เครดิตบวก หนีจากที่เครดิตลบ — แต่ไม่เกิน 20% ต่อรอบ ทำซ้ำ 20 ครั้งต่อตาราง 15,000 ตาราง

ตัวเลขทุกตัวคำนวณและตรวจสอบด้วย Python 2026-09-11 (`scripts/toy_answer_key.py` — รับจากรากของ repo ได้เลย ทุกช่อง ทุกระดับ รวมคู่ แพลตและ KL) · สูตรตรงกับ `ppo.py`, `rollout_storage.py`, `actor_critic.py` ใน `tron1-rl-isaacsim/rl1/` · เวอร์ชัน 8 มีติของช่อง e0 t0 อยู่หน้า 06 §3.5.5